705.641.81: Natural Language Processing Self-Supervised Models

Homework 6: Adapting Transformers Language Models for Your Problems

For homework deadline and collaboration policy, please see our Canvas page.

Name: Brian W. Onieal

Collaborators, if any: None

Sources used for your homework, if any: Claude (Anthropic), as described in the GenAI disclosure below. C. Clark et al., "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," NAACL 2019. A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017 (the variance argument behind the scaling in 2.3). J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL 2019, and Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019 (fine-tuning learning rates, 3.1.3 and 3.2.1). E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022. X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," ACL 2021, and B. Lester, R. Al-Rfou, and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," EMNLP 2021 (for the note in 3.2.3). Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024 (the stand-in models in 3.3). OpenAI, API deprecations page, developers.openai.com/api/docs/deprecations, read October 18, 2026. The PyTorch profiler recipe and the Hugging Face transformers and peft documentation linked in the assignment.

> GenAI disclosure: Claude (Anthropic, model claude-opus-5-5) wrote the answers, derivations, and code completions in this submission, wrote the experiment and verification scripts, and ran the experiments on my laptop's GPU (NVIDIA RTX 4050 Laptop, 6 GB) through the Claude app. The exception is the BERT-base and RoBERTa-base runs in 3.1.4, which ran on an NVIDIA L4 GPU in a Google Colab notebook that Claude wrote and I ran. I chose the scope, chose the local models that stand in for the retired OpenAI models in Section 3.3, and reviewed the work before submitting. The same disclosure is at the top of each `.py` file in the repository.
>
> Code: https://github.com/brianonieal/NLP-Assignment-6

This assignment will return to basics of Transformers and dig deeper into the fundamental concepts. Additionally, we will get our hands dirty by fine-tuning Transformer language models and comparing their performance.

Homework goals: After completing this homework, you should be comfortable with:

- Training a pre-trained Language Model for your tasks
- You will explore different variations of model training
- You will compare the above results with few-shot prompting of models.

Concepts, intuitions and big picture

1. How does the BERT model expect a pair of sentences to be processed?

☐ `Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`

☐ `[CLS] Tokens-of-sentence-1 Tokens-of-sentence-2`

☐ `[CLS] Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`

> Answer: the third option, `[CLS] Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`. The tokenizer also puts a second `[SEP]` after sentence 2, and it sets `token_type_ids` to 0 for the first segment and 1 for the second. I checked with the `bert-base-uncased` tokenizer on "The cat sat on the mat." and "It was very comfortable." It returns `[CLS] the cat sat on the mat . [SEP] it was very comfortable . [SEP]`, with type 0 on the first 9 tokens and type 1 on the last 6. In pretraining, the final hidden state at `[CLS]` feeds next sentence prediction, and `[SEP]` marks where a segment ends. The first option has no `[CLS]`. The second has no separator, so the model can't tell where sentence 1 stops.

2. When should you train a new tokenizer?

☐ When your dataset is similar to that used by an existing pretrained model, and you want to pretrain a new model

☐ When your dataset is similar to that used by an existing pretrained model, and you want to fine-tune a new model using this pretrained model

☐ When your dataset is different from the one used by an existing pretrained model, and you want to pretrain a new model

> Answer: the third option. A new tokenizer only makes sense when I'm pretraining a new model, and it only pays off when my text is different from what the existing tokenizer was built on (another language, code, or a specialized domain). If the data is similar, the existing tokenizer already splits it well. If I fine-tune an existing model, I have to keep its tokenizer. The model's embedding matrix is indexed by that tokenizer's IDs, so a new vocabulary would point every ID at the wrong embedding row. Question 6 shows what that looks like.

3. Select the sentences that apply to the BPE model of tokenization.

☐ BPE is a subword tokenization algorithm that starts with a small vocabulary and learns merge rules.

☐ BPE is a subword tokenization algorithm that starts with a big vocabulary and progressively removes tokens from it.

☐ BPE tokenizers learn merge rules by merging the pair of tokens that is the most frequent.

☐ A BPE tokenizer learns a merge rule by merging the pair of tokens that maximizes a score that privileges frequent pairs with less frequent individual parts.

☐ BPE tokenizes words into subwords by splitting them into characters and then applying the merge rules.

☐ BPE tokenizes words into subwords by finding the longest subword starting from the beginning that is in the vocabulary, then repeating the process for the rest of the text.

> Answer: the first, third, and fifth options. BPE starts from a small base vocabulary (the characters, or the 256 bytes in byte-level BPE) and learns merge rules. Each merge joins the most frequent adjacent pair. To tokenize a word, it splits the word into characters and replays the merges in the order they were learned. The other three describe other algorithms. Starting big and pruning is Unigram. Scoring pairs by $\text{count}(ab)/\big(\text{count}(a)\,\text{count}(b)\big)$, which favors frequent pairs with rarer parts, is how WordPiece trains. Longest match first from the start of the word is how WordPiece encodes.
>
> I checked this by training a BPE tokenizer with the Hugging Face `tokenizers` library on a toy corpus: hug ×10, pug ×5, pun ×12, bun ×4, hugs ×5. The vocabulary starts as the 7 letters. Before any merge, the adjacent pair counts are ug 20, pu 17, un 16, hu 15, gs 5, and bu 4, and the first merge is "u g", the top pair. The merges it learned, in order, are u g, u n, h ug, p un, p ug, and hug s. Each one was the most frequent pair at its step. For example, after "u g" the counts are un 16, h ug 15, and p u 12, so "u n" is next. The word "bug" never appeared in training, and it encodes as b + ug, which is the characters plus the merges that apply.

4. What are the labels in a masked language modeling problem?

☐ Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens.

☐ Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens, shifted to the left.

☐ Some of the tokens in the input sentence are randomly masked, and the label is whether the sentence is positive or negative.

☐ Some of the tokens in the two input sentences are randomly masked, and the label is whether the two sentences are similar or not.

> Answer: the first option. Some input tokens are masked, and the label at each masked position is the original token. The loss is cross-entropy over the vocabulary at the masked positions only. In Hugging Face the other positions get the label -100, which the loss skips. Shifting the labels left is causal language modeling, where each position predicts the next token. The last two options are sentiment and sentence-pair classification, which are fine-tuning tasks with labels a person supplies. I checked with `DataCollatorForLanguageModeling` on `bert-base-uncased` at the default masking rate of 0.15. On a 28-token sentence it picked 3 positions. Two became `[MASK]` and one became a random token ("university" became "moniker", the 10% random-replacement case from BERT). All 3 labels were the original tokens, and the other 25 labels were -100.

5. When should you pretrain a new model?

☐ When there is no pretrained model available for your specific language

☐ When you have lots of data available, even if there is a pretrained model that could work on

it

☐ When you have concerns about the bias of the pretrained model you are using

> Answer: the first and third options. With no pretrained model for my language, there's nothing to fine-tune, and a tokenizer built for another language would chop the text into tiny pieces. If I'm worried about a model's bias, fine-tuning on a small dataset doesn't undo what it learned in pretraining. Pretraining on data I chose is how I'd control that. The second option is the wrong reason. If a pretrained model already fits my data, fine-tuning it is far cheaper and keeps what it learned. Having a lot of data isn't enough reason on its own to pay for pretraining from scratch.

6. Is there something wrong with the following code?

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
model = AutoModel.from_pretrained("gpt2")

encoded = tokenizer("Hey!", return_tensors="pt")
result = model(**encoded)
```

☐ No, it seems correct.

☐ The tokenizer and model should always be from the same checkpoint.

☐ It's good practice to pad and truncate with the tokenizer as every input is a batch.

> Answer: the second option. The tokenizer and the model come from different checkpoints. I ran the code. The `bert-base-cased` tokenizer turns "Hey!" into the IDs [101, 4403, 106, 102], which mean `[CLS]`, `Hey`, `!`, `[SEP]` in BERT's 28,996-token vocabulary. GPT-2 looks the same IDs up in its own 50,257-token vocabulary and reads "¨", " Big", "®", "©". GPT-2's own tokenizer would have given [10814, 0]. The code doesn't crash, which is the dangerous part. All four IDs are below 50,257, and `GPT2Model` accepts the `token_type_ids` the BERT tokenizer adds (it looks them up in its word embeddings, so it adds the embedding of GPT-2 token 0 to every position). It returns a (1, 4, 768) tensor of hidden states for the wrong text. The third option isn't the problem here. A single sentence doesn't need padding or truncation.

Getting Your Attention with Self-Attention

Recall that the transformer architecture uses scaled dot-product attention to compute *attention weights*:

$$\boldsymbol{\alpha}^{(t)} = \text{softmax}\left(\frac{\mathrm{q}_t \mathrm{K}^\top}{\sqrt{h}}\right) \in [0,1]^n$$

The resulting embedding in the output of attention at position t are:

$$\text{Attention}(\text{Q,K,V})_t = \sum_{t\prime=1}^{n} \alpha_{t\prime}^{(t)} \text{v}_{t\prime} \in \mathbb{R}^{1\times h},$$

where $\boldsymbol{\alpha}^{(t)} = \left[\alpha_0^{(t)}, \ldots, \alpha_n^{(t)}\right]$. The same idea can be stated in a matrix form,

$$\text{Attention}(\text{Q,K,V}) = \text{softmax}\left(\frac{\text{QK}^\top}{\sqrt{h}}\right)\text{V} \in \mathbb{R}^{n\times h}.$$

In the above equations:

- h is the hidden state dimension and n is the input sequence length;
- $\text{X} \in \mathbb{R}^{n\times h}$ is the input to the attention;
- $\text{x}_t \in \mathbb{R}^{1\times h}$ is the slice of X at position t, i.e. vector representation (embedding) of the input token at position t;
- $\text{W}_q, \text{W}_k, \text{W}_v \in \mathbb{R}^{h\times h}$ are the projection matrices to build query, key and value representations;
- $\text{Q} = \text{XW}_q \in \mathbb{R}^{n\times h}$,$\text{K} = \text{XW}_k \in \mathbb{R}^{n\times h}$,$\text{V} = \text{XW}_v \in \mathbb{R}^{n\times h}$ are the query, key and value representations;
- $\text{q}_t = \text{x}_t\text{W}_q \in \mathbb{R}^{1\times h}$ is the slice of Q at position t. Similarly, $\text{k}_t = \text{x}_t\text{W}_k \in \mathbb{R}^{1\times h}$ and $\text{v}_t = \text{x}_t\text{W}_v \in \mathbb{R}^{1\times h}$.

All of the numbers in my Section 2 answers are checked in `verification/verify_section2.py`, and its output is in `verification/verify_section2_output.txt`.

Forget 'Softmax', 'Argmax' is the New Boss in Town!

Recall from lectures that we can think about attention as being queryable softmax pooling. In this problem, we ask you to consider a hypothetical 'argmax' version of attention where it returns exactly the value corresponding to the key that is most similar to the query, where similarity is measured using the traditional inner-product. In other words, here use a version of Attention that instead of using softmax we use argmax to generate outputs from the attention layer. For example, `softmax([1, 3, 2])` becomes `argmax([1, 3, 2]) = [0, 1, 0]`.

1. Perform argmax attention with the following keys and values:

$$\text{Keys} = \left\{\begin{bmatrix}0\\1\\2\end{bmatrix}, \begin{bmatrix}1\\1\\2\end{bmatrix}, \begin{bmatrix}-1\\1\\-2\end{bmatrix}, \begin{bmatrix}0\\-1\\-2\end{bmatrix}\right\},\ \text{Values} = \left\{\begin{bmatrix}3\\2\\-4\end{bmatrix}, \begin{bmatrix}3\\-2\\4\end{bmatrix}, \begin{bmatrix}3\\2\\0\end{bmatrix}, \begin{bmatrix}3\\2\\4\end{bmatrix}\right\}$$

- using the following query:

$$q = \begin{bmatrix} -2 \\ 1 \\ 1 \end{bmatrix}$$

- What would be the output of the attention layer for this query? Remember, to simplify calculations, use an argmax instead of softmax. Note about notation: each vector in 'Keys = {.}' and 'Values{.}' are key/value vectors corresponding to four, non-interchangeable positions (i.e., ordering of these vectors matter).

Answer: the output is $\mathbf{v}_1 = [3,2,-4]^\top$. Step 1 is the score of the query against each key:

$$\begin{aligned} \mathbf{q}^\top \mathbf{k}_1 &= (-2)(0) + (1)(1) + (1)(2) = 3 \\ \mathbf{q}^\top \mathbf{k}_2 &= (-2)(1) + (1)(1) + (1)(2) = 1 \\ \mathbf{q}^\top \mathbf{k}_3 &= (-2)(-1) + (1)(1) + (1)(-2) = 1 \\ \mathbf{q}^\top \mathbf{k}_4 &= (-2)(0) + (1)(-1) + (1)(-2) = -3 \end{aligned}$$

Step 2 is the weights. Scaling by $\sqrt{h} = \sqrt{3}$ divides every score by the same positive number, giving $[1.732,0.577,0.577,-1.732]$, so it doesn't change which one is largest. The largest score is the first, so argmax$([3,1,1,-3]) = [1,0,0,0]$.

Step 3 is the weighted sum of the values:

$$\text{Attention} = 1 \cdot \begin{bmatrix} 3 \\ 2 \\ -4 \end{bmatrix} + 0 \cdot \begin{bmatrix} 3 \\ -2 \\ 4 \end{bmatrix} + 0 \cdot \begin{bmatrix} 3 \\ 2 \\ 0 \end{bmatrix} + 0 \cdot \begin{bmatrix} 3 \\ 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ -4 \end{bmatrix}$$

For comparison, scaled softmax attention on the same inputs gives weights $[0.602,0.190,0.190,0.019]$ and the output $[3,1.241,-1.573]^\top$, a blend of all four values.

2. How does this design choice affect our ability to usefully train models involving attention? (**Hint:** think about how the gradients flow through the network in the backward pass. Can we learn to improve our queries or keys during the training process?)

Answer: with argmax, no gradient reaches the queries or the keys, so $\mathbf{W}_q$ and $\mathbf{W}_k$ can't learn which positions to attend to. Only the one selected value gets a gradient. Here's the derivation. Write the scores as $s_i = \mathbf{q}^\top \mathbf{k}_i / \sqrt{h}$, the weights as $\boldsymbol{\alpha} = \text{onehot}(\text{argmax}_i s_i)$, and the output as $\mathbf{o} = \sum_i \alpha_i \mathbf{v}_i$. By the chain rule, for a loss L,

$$\frac{\partial L}{\partial \mathbf{q}} = \sum_i \frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial \mathbf{q}}, \qquad \frac{\partial L}{\partial s_i} = \sum_l \frac{\partial L}{\partial \mathbf{o}} \mathbf{v}_l \frac{\partial \alpha_l}{\partial s_i}.$$

The weights are piecewise constant in the scores. If I nudge any s_i by a small enough amount, the largest score is still the largest, so $\boldsymbol{\alpha}$ doesn't change. That makes $\partial \alpha_l / \partial s_i = 0$ everywhere except at ties, where $\boldsymbol{\alpha}$ jumps and has no derivative. So $\partial L / \partial s_i = 0$, and then $\partial L / \partial \mathbf{q} = 0$ and $\partial L / \partial \mathbf{k}_i = 0$, which means $\partial L / \partial \mathbf{W}_q = \partial L / \partial \mathbf{W}_k = 0$ through this layer. The value path still works, since $\partial \mathbf{o} / \partial \mathbf{v}_j = \alpha_j \mathbf{I}$. That's $\mathbf{I}$ for the selected position j^* and zero for every other position.

Softmax doesn't have this problem. Its Jacobian is $\partial\alpha_l / \partial s_i = \alpha_l(\delta_{li} - \alpha_i)$, which isn't zero unless the weights are already one-hot. I checked both in PyTorch on the numbers from part 1, using the sum of the output as the loss. With argmax, autograd has no path back to $\mathbf{q}$ or $\mathbf{K}$ (their gradients come back as None), and the gradient on $\mathbf{V}$ is $[1,1,1]$ in the first row and zero in the other three. With scaled softmax, the gradient on $\mathbf{q}$ is $[0, -0.138, -1.297]$, and every key gets a nonzero gradient.

Which key a query matches can still change during training, because the layer's inputs change as earlier layers learn through the value and residual paths. But no gradient ever pushes a query or key toward a better match. The model can still learn what to say (the values), but not where to look. It also can't blend information from several positions, since every output is a copy of a single value.

Superposition of Information in Self-Attention

We will show that the attention mechanism is able to copy the information from its input, whenever needed.[1]

1. **'Copying' mechanism in self-attention:** We'll first show that it is particularly simple for attention to "copy" a value vector to the output. Describe (in 1-2 sentences) what properties of the inputs to the attention operation would result in the output $\text{Attention(Q,K,V)}_t$ being approximately equal to v_j for some $j \in [0 \dots n]$.
   Hint: Show that there can exist $j \in [0 \dots n]$ such that $\alpha_j^{(t)} \gg \sum_{i \neq j} \alpha_k^{(t)}$, if certain property holds for the query q_t, and the keys $\{\text{k}_0 \dots \text{k}_n\}$.

   Answer: if the query's dot product with key $\mathbf{k}_j$ is much larger than its dot product with every other key (by a margin well above $\sqrt{h}\ln n$), softmax puts almost all of the weight on position j and the output is about $\mathbf{v}_j$. One way to get that is a query that points along $\mathbf{k}_j$ with a large norm, for example $\mathbf{q}_t = c\,\mathbf{k}_j$ with large c when the keys are close to orthogonal.

   Here's the work behind it. Let $\Delta = \mathbf{q}_t\mathbf{k}_j^\top - \max_{i \neq j}\mathbf{q}_t\mathbf{k}_i^\top$ be the margin. The softmax normalizer cancels in a ratio of weights, so

$$\frac{\sum_{i \neq j} \alpha_i^{(t)}}{\alpha_j^{(t)}} = \sum_{i \neq j} \exp\left(\frac{\mathbf{q}_t\mathbf{k}_i^\top - \mathbf{q}_t\mathbf{k}_j^\top}{\sqrt{h}}\right) \leq n\, e^{-\Delta/\sqrt{h}},$$

   since there are n other keys and each exponent is at most $-\Delta/\sqrt{h}$. When $\Delta \gg \sqrt{h}\ln n$ the right side is tiny, so $\alpha_j^{(t)} \gg \sum_{i \neq j} \alpha_i^{(t)}$. The weights sum to 1, so $\alpha_j^{(t)} \to 1$, and

$$\|\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V})_t - \mathbf{v}_j\| = \left\|\sum_{i \neq j} \alpha_i^{(t)} \left(\mathbf{v}_i - \mathbf{v}_j\right)\right\| \leq \left(\sum_{i \neq j} \alpha_i^{(t)}\right) \max_i \| \mathbf{v}_i - \mathbf{v}_j \| \to 0.$$

[1] Question credit: John Hewitt

With $\mathbf{q}_t = c\,\mathbf{k}_j$ and orthonormal keys, $\mathbf{q}_t\mathbf{k}_j^\top = c$ and $\mathbf{q}_t\mathbf{k}_i^\top = 0$, so $\Delta = c$. In the check script (6 orthonormal keys in $\mathbb{R}^{16}$), α_j is 0.204 at $c = 1$, 0.916 at $c = 16$, and 0.998 at $c = 32$.

2. **Extracting signals after averaging them:** Instead of focusing on just one vector v_j, attention mechanism might want to incorporate information from multiple source vectors. Consider the case where we instead want to incorporate information from two vectors v_a and v_b, with corresponding key vectors k_a and k_b. How should we combine information from two value vectors v_a and v_b? A common way to combine values vectors is to average them: $\bar{\mathrm{v}} = \frac{1}{2}(\mathrm{v}_a + \mathrm{v}_b)$. However, after such averaging it is not quite clear how to tease apart the original information in value vectors v_a and v_b. Unless ... some special properties hold!
   Suppose that although we don't know v_a or v_b, we do know that v_a lies in a subspace[2] A formed by the m basis vectors[3] $\{\mathrm{a}_1, \mathrm{a}_2, \ldots, \mathrm{a}_m\}$, while v_b lies in a subspace B formed by the p basis vectors $\{\mathrm{b}_1, \mathrm{b}_2, \ldots, \mathrm{b}_p\}$. This means that any v_a can be expressed as a linear combination of its basis vectors, as can v_b. All basis vectors have norm 1 and are orthogonal to each other. Additionally, suppose that the two subspaces are orthogonal; i.e. $\mathrm{a}_j^\top \mathrm{b}_k = 0$ for all j, k. Using the basis vectors $\{\mathrm{a}_1, \mathrm{a}_2, \ldots, \mathrm{a}_m\}$, construct a matrix M such that for arbitrary vectors $\mathrm{v}_a \in A$ and $\mathrm{v}_b \in B$, we can use M to extract v_a from the average vector $\bar{\mathrm{v}}$. In other words, we want to construct M such that for any v_a, v_b, $M\bar{\mathrm{v}} = v_a$. Show that $M\bar{\mathrm{v}} = \mathrm{v}_a$ holds for your M.

   Answer: $M = 2\sum_{i=1}^{m} \mathbf{a}_i\,\mathbf{a}_i^\top = 2\mathbf{A}\mathbf{A}^\top$, where $\mathbf{A} = [\mathbf{a}_1, \ldots, \mathbf{a}_m] \in \mathbb{R}^{h\times m}$ has the basis vectors as columns. I treat vectors as columns here. With the row vectors used above, $\bar{\mathbf{v}}M$ works the same way, because M is symmetric.

   Proof. Put the $\mathbf{b}_k$ in the columns of $\mathbf{B} \in \mathbb{R}^{h\times p}$. Since $\mathbf{v}_a \in A$ and $\mathbf{v}_b \in B$, there are coefficient vectors $\mathbf{c} \in \mathbb{R}^m$ and $\mathbf{d} \in \mathbb{R}^p$ with $\mathbf{v}_a = \mathbf{A}\mathbf{c}$ and $\mathbf{v}_b = \mathbf{B}\mathbf{d}$. The two assumptions give two facts. The $\mathbf{a}_i$ are orthonormal, so $(\mathbf{A}^\top\mathbf{A})_{ij} = \mathbf{a}_i^\top\mathbf{a}_j = \delta_{ij}$, which means $\mathbf{A}^\top\mathbf{A} = \mathbf{I}_m$. The subspaces are orthogonal, so $(\mathbf{A}^\top\mathbf{B})_{jk} = \mathbf{a}_j^\top\mathbf{b}_k = 0$, which means $\mathbf{A}^\top\mathbf{B} = \mathbf{0}$. Then

$$M\bar{\mathbf{v}} = 2\mathbf{A}\mathbf{A}^\top \cdot 1/2\,(\mathbf{A}\mathbf{c} + \mathbf{B}\mathbf{d}) = \mathbf{A}(\mathbf{A}^\top\mathbf{A})\mathbf{c} + \mathbf{A}(\mathbf{A}^\top\mathbf{B})\mathbf{d} = \mathbf{A}\mathbf{I}_m\mathbf{c} + \mathbf{A}\mathbf{0}\,\mathbf{d} = \mathbf{A}\mathbf{c} = \mathbf{v}_a.$$

   $\mathbf{A}\mathbf{A}^\top$ on its own is the orthogonal projection onto A. It keeps the $\mathbf{v}_a$ half of the average and removes the $\mathbf{v}_b$ half, which leaves $1/2\,\mathbf{v}_a$. The factor of 2 undoes the averaging. M only needs the basis of A, not $\mathbf{v}_a$ or $\mathbf{v}_b$. In the check script ($h = 20$, $m = 4$, $p = 5$, random orthonormal bases), the largest error in $M\bar{\mathbf{v}} - \mathbf{v}_a$ over 1,000 random pairs was 5.6×10^{-16}, which is floating-point rounding.

3. **Averaging pairs of representations:** As before, let v_a and v_b be two value vectors corresponding to key vectors k_a and k_b, respectively. Assume that (1) all key vectors are

---

[2] https://en.wikipedia.org/wiki/Linear_subspace

[3] https://en.wikipedia.org/wiki/Basis_(linear_algebra)

orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1. Find an expression for a query vector q_t such that $\text{Attention}(Q,K,V)_t = \frac{1}{2}(v_a + v_b)$ and justify your answer.

More here: https://web.stanford.edu/class/cs224n/assignments/a5.pdf

Answer: $\mathbf{q}_t = c\,(\mathbf{k}_a + \mathbf{k}_b)$ with c large ($c \gg \sqrt{h}\ln n$). The output approaches $1/2\,(\mathbf{v}_a + \mathbf{v}_b)$ as $c \to \infty$, and the error shrinks like $e^{-c/\sqrt{h}}$.

Justification. Use the two assumptions to get every score:

$$\mathbf{q}_t\mathbf{k}_a^\top = c\,(\mathbf{k}_a\mathbf{k}_a^\top + \mathbf{k}_b\mathbf{k}_a^\top) = c\,(1+0) = c, \qquad \mathbf{q}_t\mathbf{k}_b^\top = c\,(0+1) = c, \qquad \mathbf{q}_t\mathbf{k}_i^\top = c\,(0+0) = 0 \ \text{ for } i \notin \{a,b\}.$$

With $N = n+1$ keys and $z = e^{c/\sqrt{h}}$, the softmax weights are

$$\alpha_a^{(t)} = \alpha_b^{(t)} = \frac{z}{2z + (N-2)}, \qquad \alpha_i^{(t)} = \frac{1}{2z + (N-2)} \ \text{ for } i \notin \{a,b\}.$$

α_a and α_b are equal for every c, so the balance between $\mathbf{v}_a$ and $\mathbf{v}_b$ is exact. Dividing the top and bottom by z, $\alpha_a = 1/(2 + (N-2)/z) \to 1/2$ as $c \to \infty$, and the total weight left on the other keys is $(N-2)/(2z+N-2) \leq N - 2/2\,e^{-c/\sqrt{h}} \to 0$. So

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V})_t = \alpha_a\mathbf{v}_a + \alpha_b\mathbf{v}_b + \sum_{i\notin\{a,b\}} \alpha_i\,\mathbf{v}_i \ \to\ 1/2\,(\mathbf{v}_a + \mathbf{v}_b).$$

Softmax never gives a weight of exactly 0, so for any finite c the result is approximate, with an error that falls off exponentially. In the check script (6 orthonormal keys in $\mathbb{R}^{16}$), the weight on the other four keys is 0.213 at $c = 8$, 0.00067 at $c = 32$, and under 10^{-6} at $c = 64$, and each value matches the formula above.

Extra Credit: Importance of 'Scaling' in Self-Attention

In practice, we scale each dot product $QK^\top$ by a factor of $\sqrt{h}$. This is called *scaled dot product attention*. In this part, we will prove why we perform this scaling. Suppose we are performing a dot product between a key k and query q, where $k,q \in \mathbb{R}^{1\times h}$ and $k,q \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2\mathbf{I})$. You can assume that k,q are sampled independently.

Setup for all three parts. Write $\mathbf{q}\mathbf{k}^\top = \sum_{i=1}^{h} q_i\,k_i$ and $\boldsymbol{\mu} = [\mu_1, \ldots, \mu_h]$. A covariance of $\sigma^2\mathbf{I}$ means the coordinates of $\mathbf{q}$ are independent with $q_i \sim \mathcal{N}(\mu_i, \sigma^2)$, and the same holds for $\mathbf{k}$. Since $\mathbf{q}$ and $\mathbf{k}$ are also independent of each other, all $2h$ numbers $q_1, \ldots, q_h, k_1, \ldots, k_h$ are independent. When every coordinate has the same mean μ, $\| \boldsymbol{\mu} \|^2 = h\mu^2$.

1. Compute $\mathbf{E}[qk^\top]$ in terms of $\boldsymbol{\mu}, h, \sigma$.

   Answer: $\mathbf{E}[\mathbf{q}\mathbf{k}^\top] = \boldsymbol{\mu}\boldsymbol{\mu}^\top = \| \boldsymbol{\mu} \|^2$, which is $h\mu^2$ when every coordinate has mean μ. It doesn't depend on σ.

$$\mathbf{E}[\mathbf{q}\mathbf{k}^\top] = \mathbf{E}\left[\sum_{i=1}^{h} q_i\,k_i\right] = \sum_{i=1}^{h} \mathbf{E}\,[q_i k_i] = \sum_{i=1}^{h} \mathbf{E}\,[q_i]\,\mathbf{E}[k_i] = \sum_{i=1}^{h} \mu_i \cdot \mu_i = \| \boldsymbol{\mu} \|^2.$$

The second step is linearity of expectation. The third uses the independence of q_i and k_i.

2. Compute Var$\left[\mathrm{qk}^{\top}\right]$ in terms of $\boldsymbol{\mu}, h, \sigma$.

Answer: $\mathrm{Var}[\mathbf{qk}^{\top}] = h\sigma^4 + 2\sigma^2 \parallel \boldsymbol{\mu} \parallel^2$, which is $h\sigma^4 + 2h\sigma^2\mu^2$ when every coordinate has mean μ.

Step 1. The terms $q_i k_i$ are independent of each other, because term i uses only q_i and k_i and all $2h$ numbers are independent. The variance of a sum of independent terms is the sum of their variances:

$$\mathrm{Var}\left[\sum_{i=1}^{h} q_i\, k_i\right] = \sum_{i=1}^{h} \mathrm{Var}\,[q_i k_i].$$

Step 2. One term. Use $\mathrm{Var}[X] = \mathbf{E}[X^2] - \mathbf{E}[X]^2$, then independence to split $\mathbf{E}[q_i^2 k_i^2] = \mathbf{E}[q_i^2]\,\mathbf{E}[k_i^2]$, and $\mathbf{E}[q_i^2] = \mathrm{Var}[q_i] + \mathbf{E}[q_i]^2 = \sigma^2 + \mu_i^2$:

$$\mathrm{Var}[q_i k_i] = \mathbf{E}[q_i^2]\,\mathbf{E}[k_i^2] - (\mathbf{E}[q_i]\,\mathbf{E}[k_i])^2 = (\sigma^2 + \mu_i^2)^2 - \mu_i^4 = \sigma^4 + 2\sigma^2\mu_i^2.$$

Step 3. Add up the h terms:

$$\mathrm{Var}[\mathbf{qk}^{\top}] = \sum_{i=1}^{h} (\sigma^4 + 2\sigma^2\mu_i^2) = h\sigma^4 + 2\sigma^2 \parallel \boldsymbol{\mu} \parallel^2.$$

A Monte Carlo check with 400,000 draws matches both parts. With $h = 64$, $\mu = 0.5$, $\sigma = 2$, the sample mean is 16.03 against $h\mu^2 = 16$, and the sample variance is 1,146 against $h\sigma^4 + 2h\sigma^2\mu^2 = 1{,}152$. With $h = 16$, $\mu = -1$, $\sigma = 0.5$, I got 15.99 against 16 and 9.00 against 9.

3. Based on the variance computed above and assuming that $\boldsymbol{\mu} = 0$ and $\sigma = 1$, explain why we need to scale the dot product by $\sqrt{h}$. Explain why this scaling is important for numerical stability.

Answer: with $\boldsymbol{\mu} = 0$ and $\sigma = 1$, part 2 gives $\mathrm{Var}[\mathbf{qk}^{\top}] = h$, so a raw attention score has a standard deviation of $\sqrt{h}$. The scores spread out as the dimension grows. For a 64-dimensional head (BERT-base's 768 split over 12 heads) the standard deviation is 8, and for $h = 768$ it's about 27.7. Dividing by $\sqrt{h}$ fixes the variance at 1 for any h:

$$\mathrm{Var}\left[\frac{\mathbf{qk}^{\top}}{\sqrt{h}}\right] = \frac{\mathrm{Var}[\mathbf{qk}^{\top}]}{h} = \frac{h}{h} = 1.$$

That matters because softmax saturates when its inputs are far apart. A gap of 8 between two scores already makes one weight $e^8 \approx 3{,}000$ times the other. Then the weights are nearly one-hot, and the softmax Jacobian $\mathrm{diag}(\boldsymbol{\alpha}) - \boldsymbol{\alpha}\boldsymbol{\alpha}^{\top}$ goes to zero, so the queries and keys stop getting gradient. That's the argmax problem from 2.1, approached gradually. In the check script I drew one query and 32 keys from $\mathcal{N}(0, \mathbf{I})$ and averaged over 2,000 trials. Unscaled, the largest weight averaged 0.815 at $h = 64$ and 0.954 at $h = 1024$, and the Frobenius norm of the Jacobian fell from 0.179 to 0.061. Scaled, the largest weight stayed near 0.16 and the Jacobian norm stayed near 0.24 at every h from 4 to 1,024.

For numerical stability, large scores also overflow e^x. The largest float16 number is 65,504, so e^x overflows in float16 once $x > \ln 65{,}504 \approx 11.09$ (float32 lasts until about 88.7). With $h = 64$, the largest of 32 raw scores averaged 16.5, and e^x of it overflowed float16 in 92% of trials. At $h = 256$ it overflowed in every trial. Softmax code usually subtracts the max score before exponentiating, which prevents the overflow. It doesn't prevent the saturation, though, and it doesn't stop small weights from rounding to zero at low precision. Keeping the scores at unit scale handles both. Vaswani et al. (2017) give the same variance argument for dividing by $\sqrt{d_k}$.

Programming

In this programming homework, we will finetune encoder LMs for a classification task with yes/no questions. In particular, we will

- Implement full-parameter finetuning.
- Explore various parameter-efficient finetuning strategies.

Skeleton Code and Structure:

The code base for this homework can be found at this GitHub repo under the `hw6` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `base_classification.py`: implements the basic full finetuning
- `classification.py`: implements different parameter-efficient finetuning

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file, which includes the two provided skeleton files and a `openai_gpt-boolq.py` script you created (details in 3.3).

The code is in the repository linked at the top and in `hw6_code.zip`. Every training run used the same setup unless I say otherwise: my laptop's NVIDIA RTX 4050 Laptop GPU (6 GB), Python 3.11, PyTorch 2.2.1, transformers 4.38.1, and datasets 2.17.1, with the skeleton's max_len of 128, AdamW, and a linear schedule with 50 warmup steps. I made five changes outside the TODOs, and each one is marked in the code. (1) Both scripts shuffle BoolQ with a fixed seed (42), so every run sees the same 8,000 training examples

and the same 1,427-example test split. The skeleton's unseeded shuffle drew a new split every run, which would make the grid comparisons noisy. (2) In `classification.py` the learning-rate scheduler was attached to the full-model optimizer, but head and prefix tuning step `custom_optimizer`, so their learning rate never warmed up or decayed. The scheduler now wraps whichever optimizer steps. (3) `classification.py` prints the number of parameters it updates. (4) `classification.py` gives the `--type auto` model a `.type` attribute, because `train()` reads it and the skeleton crashed without it. (5) All three training scripts take an `--amp` flag that runs the forward pass in bfloat16 mixed precision. It's off by default. The GPU was already at 90 to 100% use in float32, so to finish sooner I turned it on for the prefix-tuning run, the LoRA run, and the 3.1.4 runs. Every other run is float32. The dev set is BoolQ's 3,270-question validation split. On dev, 2,033 answers (62.2%) are yes, and on test, 889 (62.3%) are, so a model that always says yes scores 0.622 on dev and 0.623 on test. Logs for every run are in `logs/`, and the parsed numbers are in `results/results.json`.

Overview and Dataset

In this programming assignment, you will get your hands dirtier with building self-supervised models. You will build a classifier using the Huggingface Transformer library and PyTorch. This classifier is expected to solve the BoolQ dataset. You can find this dataset on Huggingface's dataset hub: https://huggingface.co/datasets/boolq. In this task, each input consists of a question statement and a passage that may contain the answer to this question. The output is the answer to the question which may be one of 'Yes' or 'No' labels. Here is an example:

```
{
    "passage": "\"All biomass goes through at least some of these steps: it
needs to be grown, collected, dried, fermented, distilled, and burned...",
    "question": "does ethanol take more energy make that produces",
    "answer": false,
}
```

The data comes with training and validation subsets. We will select a subset of the training data as 'test' set. So for the main experiments we will use 8k instances for training and the rest for testing.

Python Scripts Arguments

Many of the **TODOs** below involves setting different training configurations and hyper-parameters, this could be achieved through the arguments provided in `main` of both `base_classification.py` and `classification.py` - they should be self-explanatory.

General Hint

Follow the specifications and requirements in the code comments to complete missing lines.

Finetuning: Basics

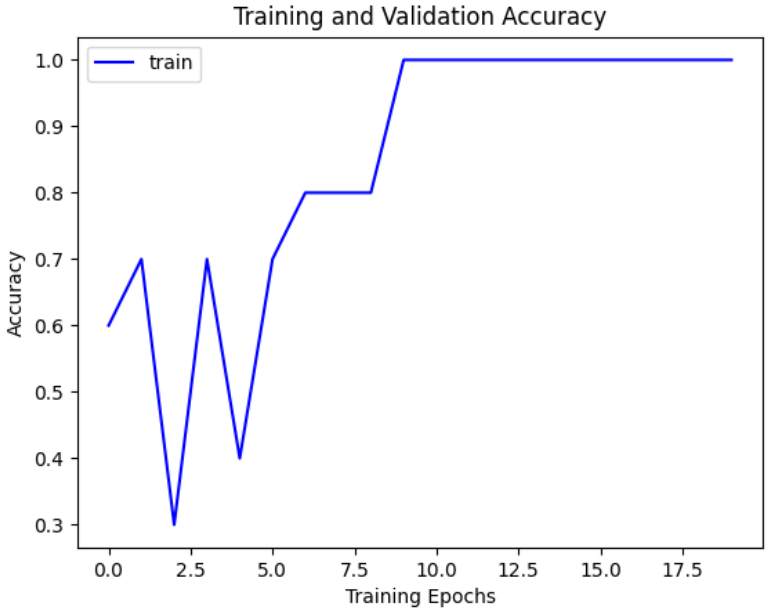
1. Let's start with the basic full finetuning, where all the parameters will be updated during the training. Specifically, we will use `AutoModelForSequenceClassification` which implements a classifier that maps the representation of encoder LMs (BERT, RoBERTa, etc.) into a fixed set of labels. Use this class to build a binary classifier using the representations of `distilbert-base-uncased`. The starter code for full finetuning in `base_classification.py` has a bunch of basic functionalities implemented, but it is missing details.
   TODOs: Read and complete the missing lines in `evaluate_model` and `train` functions in `base_classification.py` to complete the functionalities of full finetuning.

   Done in `base_classification.py`. `evaluate_model` moves `input_ids` and `attention_mask` to the device and runs the forward pass under `torch.no_grad()`, since evaluation needs no gradients. The training loop moves the inputs and labels to the device, takes `output.logits` from the model, computes the cross-entropy loss, calls `backward()`, steps the optimizer and the scheduler, and zeroes the gradients. I named the loss value `model_loss`, because the skeleton already uses `loss` for the `CrossEntropyLoss` object, and overwriting it would break the second batch.

- To make sure your implementation is correct, we can train the model on 10 training instances for a few epochs (say, a total of 20 epochs). Note that over-fitting a few training examples is a good first test to make sure your implementation can successfully learn the patterns of your data. You should be able to complete the starter code and run it on a few instances in your local machine. You should be able to see that your classifier successfully over-fits its few training examples.
  TODOs: Run `base_classification.py` with the following command on your local machine, paste the generated plot of training accuracy vs. epochs plots below.

  > python base_classification.py –small_subset –device cuda –model "distilbert-base-uncased" –batch_size "64" –lr 1e-4 –num_epochs 20

  The overfit check passes. I ran the command above as given. Training accuracy is measured during each epoch, with dropout on, over the same 10 examples. It moved between 0.3 and 0.8 for the first 9 epochs, reached 1.0 in epoch 10, and stayed at 1.0 for the last 10 epochs. The generated plot numbers epochs from 0, so epoch 10 is at x = 9. With `--small_subset` the script also evaluates on those same 10 examples in eval mode, and that reached 1.0 in epoch 6. The final dev and test accuracy it prints are both 1.0, again on those 10 examples.



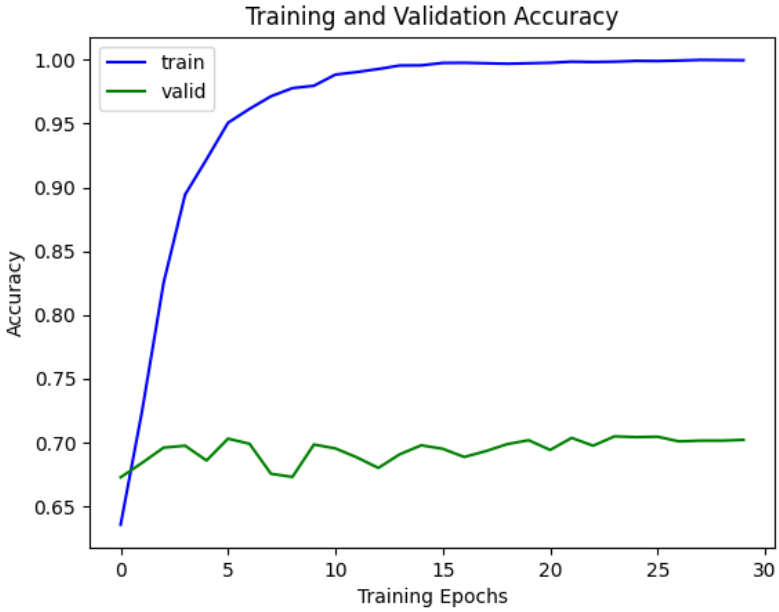
Generated plot from the 3.1.1 command (`plots/3.1.1_overfit_distilbert.png`).

2. Now, rather than running training on a subset of the data, try training with all the data on your compute environment.

- **TODOs** Train for 30 epochs and paste the generated plot of train and dev accuracies vs. epochs below. Also report the largest batch size.
  Hint: You should see that training accuracy goes high, while the dev accuracy will go high and then turn downhill as the model overfits the data.

  Answer: the largest batch size that fits is 106. I found it with `experiments/probe_batch_size.py`. It runs two real training steps of distilBERT at max_len 128 with AdamW at each batch size, doubling until a step runs out of memory and then bisecting. Batch 106 peaked at 5.89 GiB of the card's 6.00 GiB, and 107 ran out of memory. During the probe I capped PyTorch at the card's physical memory, because on Windows the driver can spill into system RAM instead of failing, and a batch that "fits" that way runs many times slower.

  For the 30-epoch run I kept batch 64 and learning rate 1e-4 from the 3.1.1 command (peak 3.96 GB reserved, 30 minutes). Training accuracy goes from 0.636 after epoch 1 to 0.951 after epoch 6 and 1.000 after epoch 28. Dev accuracy rises from 0.673 to 0.698 by epoch 4, then moves sideways between 0.673 and 0.705 for the next 26 epochs and ends at 0.702. Test accuracy after the last epoch is 0.693. So the model memorizes the training set while dev stops improving after a few epochs. That's the overfitting the hint describes. In my run it shows up as a 30-point gap between train and dev, not as a dev curve that clearly turns down. The closest thing to a downturn is a dip to 0.676 and 0.673 in epochs 8 and 9. I think the linear learning-rate decay is part of why dev doesn't fall further. By the time train accuracy is near 1.0, the steps are small, so the weights don't move far from where they were.



Generated plot from the 30-epoch run (`plots/3.1.2_full30_distilbert.png`). The x-axis counts epochs from 0.

3. Let's implement hyper-parameter selection by exploring the combination of different hyper-parameter choices.

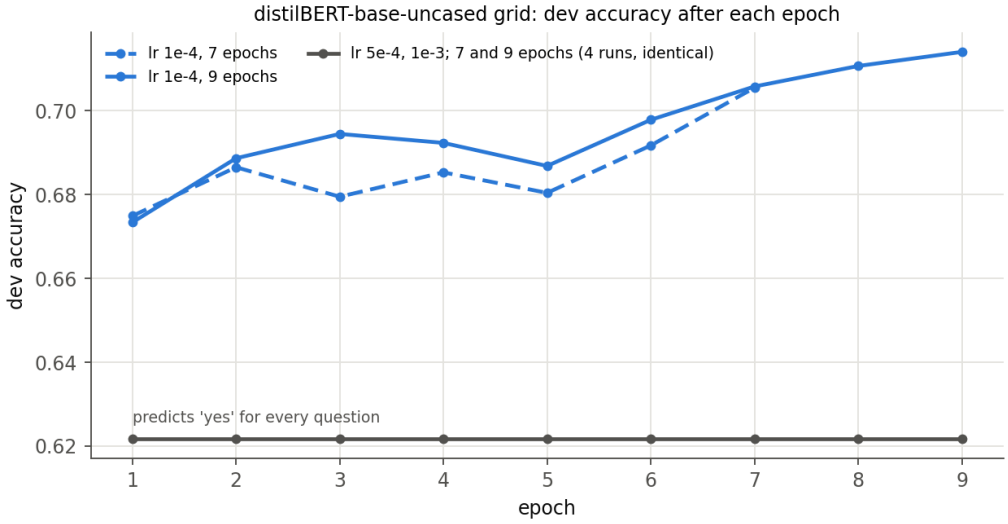
   TODOs: Train models for various choices of learning rates (`1e-4, 5e-4, 1e-3`) and training epochs (`7, 9`). Select the model with the highest accuracy on the dev set and report its accuracy on the test set. Also report the corresponding choice of hyperparameters

   Answer: the best model on dev is learning rate 1e-4 with 9 epochs (batch 64). Its dev accuracy is 0.7141 (2,335 of 3,270) and its test accuracy is 0.7057 (1,007 of 1,427). I picked on the dev accuracy after the last epoch, which is what the script reports, and only looked at test for the chosen model. All six runs used distilBERT-base-uncased, batch 64, and the same seed and split.

Learning rate	Epochs	Dev accuracy	Test accuracy
1e-4	7	0.7055	0.7050
1e-4 (selected)	9	0.7141	0.7057
5e-4	7	0.6217	0.6230
5e-4	9	0.6217	0.6230
1e-3	7	0.6217	0.6230
1e-3	9	0.6217	0.6230

   The four runs at 5e-4 and 1e-3 failed the same way. From the first epoch on they score 2,033 of 3,270 on dev and 889 of 1,427 on test, which are the exact counts of yes answers

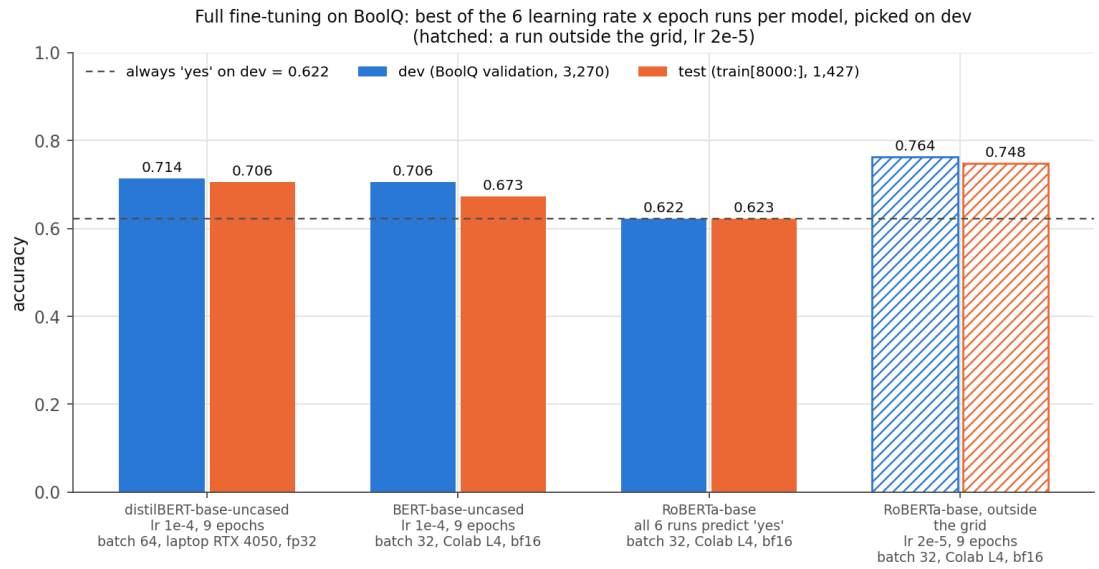
in each split, so they're predicting yes for every question. Their training accuracy sits at 0.623, the share of yes answers in the 8,000 training examples. Those learning rates are 10 to 50 times the 2e-5 to 5e-5 range the BERT paper used for fine-tuning (Devlin et al., 2019). Steps that large wreck the pretrained weights within the first epoch (125 updates), and the model falls back on the majority label. Between the two runs at 1e-4, the 9-epoch run is ahead by 0.009 on dev and 0.001 on test. That's 28 and 1 more questions right, so I wouldn't read much into the epoch choice.



Dev accuracy after each epoch for the six runs (`plots/3.1.3_grid.png`).

4. **TODOs** Now repeat the previous experiment with other models. In particular, create a bar plot that shows test/dev accuracies for two of your favorite of the following models: `distilBERT-base-uncased`, `BERT-base-uncased`, `BERT-large-uncased`, `BERT-base-cased`, `BERT-large-cased`, `RoBERTa-base`, `RoBERTa-large`. Note, it is possible that some of these models wouldn't fit in your compute environment for any choice of batch size, in which case report 0 for their performance.

   I picked BERT-base-uncased and RoBERTa-base and ran the 3.1.3 grid for each (learning rates 1e-4, 5e-4, and 1e-3, times 7 and 9 epochs) with the same `base_classification.py`, seed, and data split. Each model's bars are its best run on dev, and I only looked at test for that run. distilBERT from 3.1.3 is there for reference.



Best run per model, picked on dev (`plots/3.1.4_model_comparison.png`). The dashed line is always answering yes.

Model	Selected run	Dev accuracy	Test accuracy
distilBERT-base-uncased	1e-4, 9 epochs	0.7141	0.7057
BERT-base-uncased	1e-4, 9 epochs	0.7055	0.6734
RoBERTa-base	none, all 6 runs predict yes	0.6217	0.6230

All twelve runs for the two new models:

Learning rate	Epochs	BERT-base dev	BERT-base test	RoBERTa-base dev	RoBERTa-base test
1e-4	7	0.6869	0.6734	0.6217	0.6230
1e-4	9	0.7055	0.6734	0.6217	0.6230
5e-4	7	0.6217	0.6230	0.6217	0.6230
5e-4	9	0.6217	0.6230	0.6217	0.6230
1e-3	7	0.6217	0.6230	0.6217	0.6230
1e-3	9	0.6217	0.6230	0.6217	0.6230

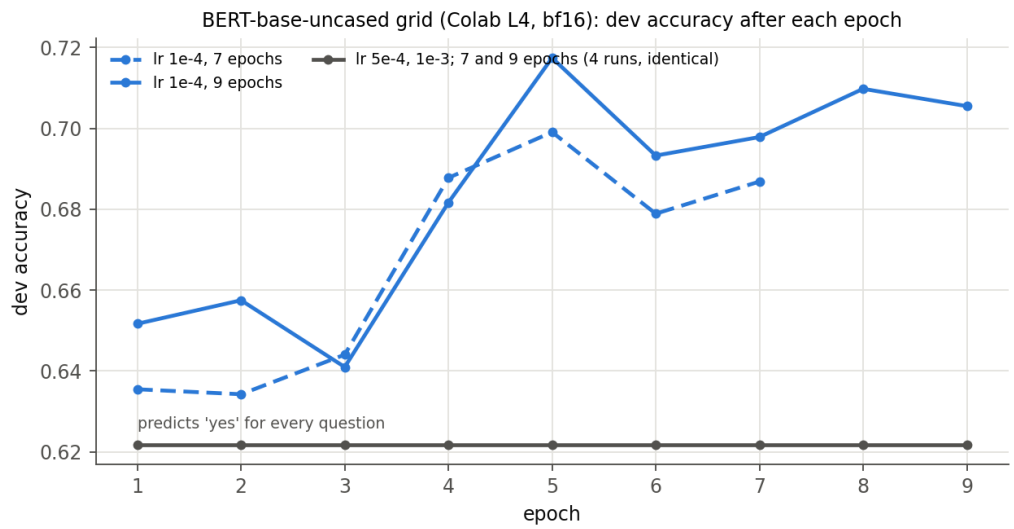
Where the runs happened. By my estimate from the 3.2.1 run, the twelve runs would have taken my laptop about three and a half hours, even in bfloat16. So I ran both grids on a Google Colab L4 GPU, where each run took 5 to 7 minutes, with bfloat16 mixed precision (`--amp`) and batch 32. Batch 32 came from my laptop's memory probe. In bfloat16 the largest batch that fit on my 6 GB card was 69 for BERT-base and 66 for RoBERTa-base, too close to 64 to be safe, and I kept 32 on Colab so the setup didn't change with the hardware. The distilBERT runs were on my laptop in float32 at batch 64. So the bars differ in hardware, precision, and batch size as well as model. To check whether bfloat16 alone moves the numbers, I reran the selected distilBERT setting (1e-4,

9 epochs, batch 64) on my laptop with `--amp`. It scored 0.7208 on dev and 0.7085 on test, against 0.7141 and 0.7057 in float32, and ran in 6.7 minutes instead of 9.2. That's within 0.7 points, so bfloat16 isn't what separates the models here.

The grid ranks the models backwards. distilBERT, the smallest, comes first, and RoBERTa-base, which beat it by 7 points in 3.2.1, comes last. That says more about the grid than about the models. Ten of the twelve new runs collapsed to always-yes from the first epoch, the same failure as distilBERT at 5e-4 and 1e-3 in 3.1.3. That's all six RoBERTa-base runs and both BERT-base learning rates above 1e-4. Even 1e-4 is twice the top of the 2e-5 to 5e-5 range the BERT paper used (Devlin et al., 2019) and more than three times the top of RoBERTa's 1e-5 to 3e-5 (Liu et al., 2019).

BERT-base at 1e-4 did learn, but it overfit fast. Training accuracy reached 0.994 by epoch 9, while dev peaked at 0.717 after epoch 5 and then moved between 0.693 and 0.710. Its selected run lands 0.9 points below distilBERT on dev and 3.2 points below on test. Both of its 1e-4 runs got exactly 961 of 1,427 test questions right.

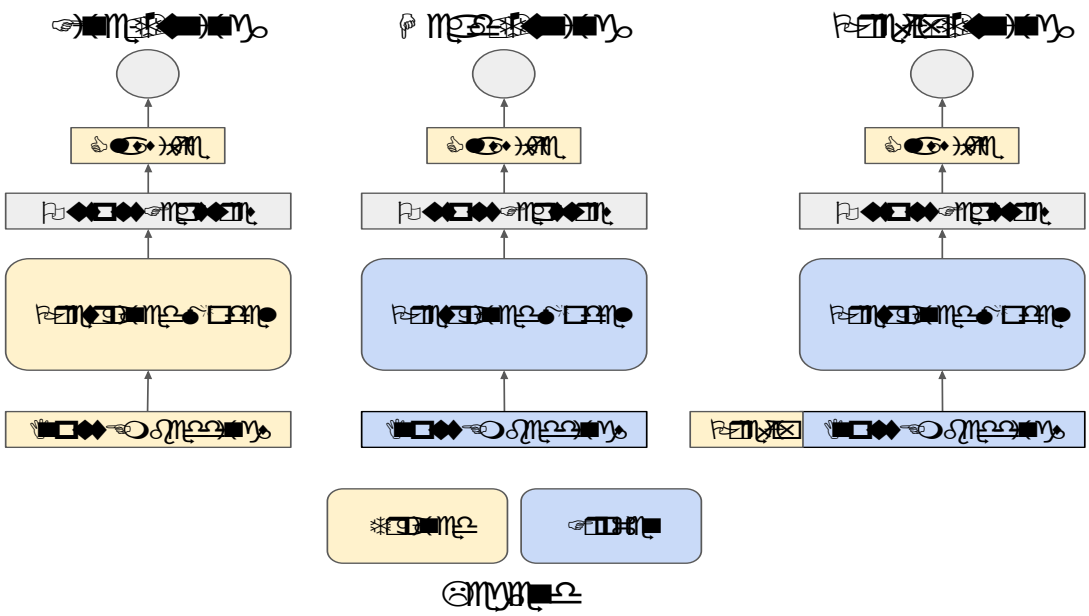
RoBERTa-base collapsed even at 1e-4. To check that bfloat16 wasn't the cause, I reran the first grid point (1e-4, 7 epochs) on the L4 in float32. It collapsed the same way, to 0.6217 on dev and 0.6230 on test. Then I ran RoBERTa-base once outside the grid, at 2e-5, inside its paper's range, for 9 epochs with everything else the same (bfloat16, batch 32). It learned. Dev accuracy rose every epoch, from 0.622 after epoch 1 to 0.764 after epoch 9, and test accuracy was 0.748. Those are the hatched bars. They put RoBERTa-base 5 points above distilBERT on dev and 4 on test. So the solid bars answer the question as asked, each model's best run in this grid, but the grid's learning rates are too high for these models. At a learning rate from its own paper, RoBERTa-base is the strongest of the three, which matches 3.2.1. This run is 2.2 points below the 3.2.1 run on dev, which also used 2e-5 but had the mean-pooled head, 6 epochs, and float32 on my laptop. Dev was still rising at epoch 9, so more epochs would likely help.



Dev accuracy after each epoch for the BERT-base grid (`plots/3.1.4_grid_bert-base-uncased.png`). All six RoBERTa-base runs sat at 0.6217 every epoch, so their plot is one flat line (`plots/3.1.4_grid_roberta-base.png`).

Parameter-Efficient Fine-Tuning

In this section, we will implement two more efficient ways of tuning language models: **Head-Tuning**, which freezes all parameters but the classification head, and **Prefix-Tuning**, which also freezes all parameters but the classification head, and additionally appends a fixed length ℓ of trainable embeddings to the input embeddings. See figure 1 for an illustration of which parameters are getting updated in each fine-tuning method we will explore in this homework. Note that "output features" tensor for a single sequence of input should be of shape $(\text{length,hidden size})$, or equivalently $\mathbb{R}^{N\times d}$. However, the classifier expects a tensor of shape $\mathbb{R}^{1\times d}$. There are multiple ways to transform the output features to $\mathbb{R}^{1\times d}$, but in this work, we will take the **averaged** token representation.
Instead of using `AutoModelForSequenceClassification`, we now use the base `AutoModel` class to tailor the model to our needs. But don't worry, a skeleton code is provided to you in `classification.py`, you would only need to modify the details.



Overview of fine-tuning methods. Blue indicates parts that are frozen (no gradient update) during the fine-tuning process. Yellow indicates parts that are trained (with gradient updates).

1. For comparison, let's redo the full finetuning first. Change the base model to `RoBERTa-base`. Implement full fine-tuning by adding a binary classifier on the averaged sentence representation.

- **TODOs**: Read and complete the missing lines

    a. in `evaluate_model` function

    b. after `for index, batch in tqdm(enumerate(train_dataloader)):` in `train` function

    in `classification.py` to complete the functionalities of full finetuning.
    Hint: Your implementation should be the same as that of `base_classification.py` for `evaluate_model` and almost the same except for how we get the logtis for `train` (until loss backward).
    TODOs: Read and complete the missing lines after `if self.type == 'full'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. Run the code with `type="full"` to train the model with full finetuning. Report the final accuracy you obtained along with the hyper-parameters you used (number of epochs and learning rate).

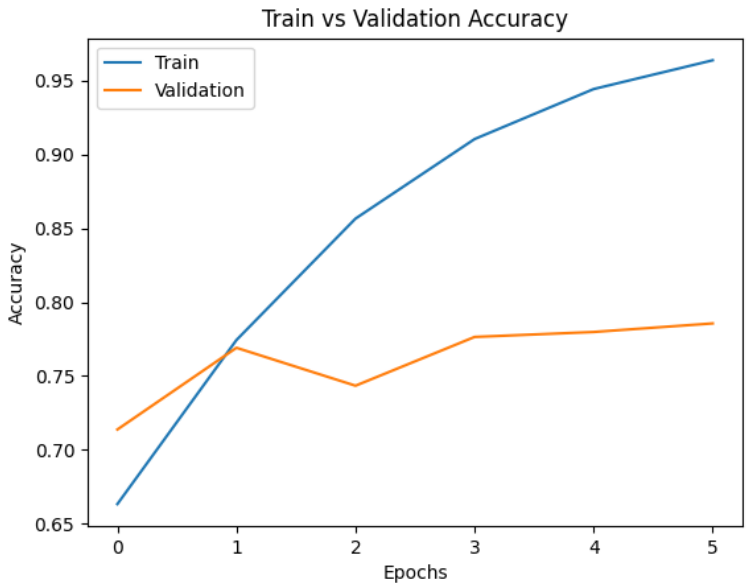
Done in `classification.py`. `evaluate_model` is the same as in `base_classification.py`. In `train`, the one difference is that `CustomModelforSequenceClassification` returns a dict, so the logits are `output['logits']` instead of `output.logits`. The `full` branch of `forward` runs RoBERTa on `input_ids` and `attention_mask`, takes `last_hidden_state` with shape (batch, 128, 768), averages it over the sequence dimension to get (batch, 768), and

passes that through `self.classifier`, a 768-to-2 linear layer, to get (batch, 2) logits. I followed the skeleton's plain mean, which also averages the padding positions. A mean weighted by the attention mask would skip them.

Answer: dev accuracy 0.7856 and test accuracy 0.7730, with learning rate 2e-5, 6 epochs, and batch 32 (17 minutes on my GPU). I chose 2e-5 because it's in the 1e-5 to 3e-5 range the RoBERTa paper used for fine-tuning (Liu et al., 2019), and the 3.1.3 grid had already shown that 5e-4 and up collapses to always-yes. Dev accuracy after each epoch was 0.714, 0.769, 0.743, 0.776, 0.780, and 0.786, and training accuracy reached 0.964 in the last epoch.

That's 7.2 points above the best distilBERT model from 3.1.3 on dev (0.786 against 0.714) and 6.7 points on test (0.773 against 0.706). RoBERTa-base is about twice distilBERT's size and was pretrained on more text, but this isn't a clean model-only comparison. The head is different (an average over tokens here, the `[CLS]` vector in `AutoModelForSequenceClassification`), and so are the batch size and learning rate.

The script printed 124,745,474 tuned parameters. That count includes the 128 × 768 = 98,304-number prefix tensor, which every `CustomModelforSequenceClassification` creates but full mode never uses, so it gets no gradient and AdamW never changes it. RoBERTa's pooler is in the same position. `AutoModel` loads RoBERTa with it, a 768 × 768 layer plus bias (590,592 numbers) that only feeds the `[CLS]`-based `pooler_output`, and the mean-pooled head never uses that output. So the parameters that actually train are RoBERTa-base's 124,645,632 minus the pooler's 590,592, plus the 1,538 in the head, 124,056,578 in total.



Generated plot from the full fine-tuning run (`plots/3.2.1_roberta_full.png`).

2. **TODOs**: Write down the number of tuned parameters for head-tuning for BoolQ, given that the model hidden size is d.

Answer: $2d + 2$. Head-tuning only trains `self.classifier`, a linear layer from d to 2 labels. Its weight matrix is $2 \times d$, which is $2d$ numbers, and its bias has 2 more. For RoBERTa-base, $d = 768$, so that's $2(768) + 2 = 1{,}538$. The training script prints the count of what its optimizer updates, and it printed 1,538 for the head-tuning run.

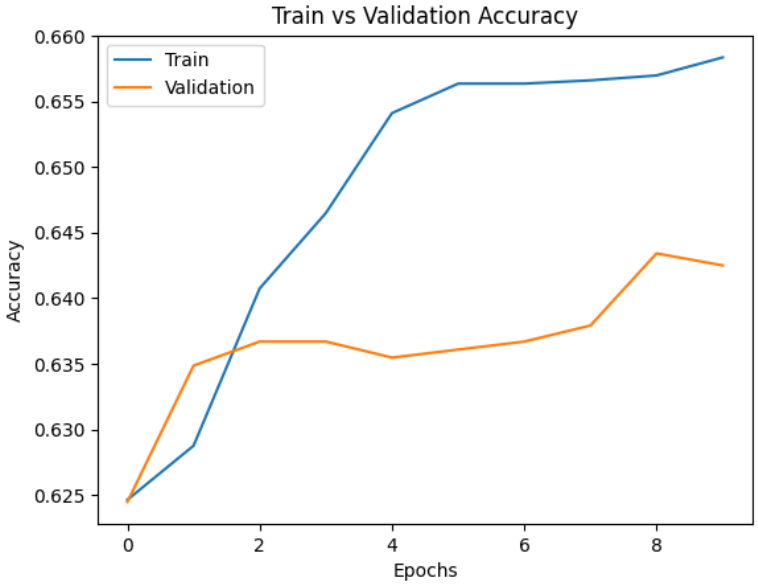
- Now let's implement head-tuning by adding a binary classifier on the averaged sentence representation.
  TODOs: Read and complete the missing lines after `if self.type == 'head'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. You would also need to modify the optimizer by adding code after `mymodel.type == "head":` in the `train` function in `classification.py`. Report the final accuracy on full head-tuning along with the hyper-parameters you used[4].

  Done in `classification.py`. The `head` branch of `forward` is the same as `full`. The difference is in `train()`, where I freeze the encoder (`requires_grad = False` on every RoBERTa parameter) and build `custom_optimizer`, an AdamW over the classifier's weight and bias only, with weight decay on the weight and not the bias, the same rule as the full optimizer. Freezing matters for more than speed. Without it, `backward()` still computes gradients for all of RoBERTa every step. They take as much memory as in full fine-tuning, and since `custom_optimizer.zero_grad()` only clears the head, they keep adding up without ever being used. The skeleton's comments say `mymodel.head`, but the attribute is `mymodel.classifier`.

  Answer: dev accuracy 0.6425 and test accuracy 0.6391, with learning rate 1e-3, 10 epochs, and batch 32. Following the footnote, I used a 50 times larger learning rate and more epochs than full fine-tuning. Only 1,538 parameters train. Dev accuracy crept from 0.624 after epoch 1 to 0.643 after epoch 9, and training accuracy only reached 0.658. That's 2 points above always-yes on dev (0.622) and 14 points below full fine-tuning.

  The low training accuracy is the telling part. The head can't even fit the training set, so the limit is the frozen features, not the learning rate or the number of epochs. A BoolQ answer depends on how the question lines up with one specific part of the passage. An average of frozen RoBERTa token vectors over all 128 positions (padding included) doesn't carry that comparison, and a single linear layer can't recover it. The encoder has to change for that, which is what full fine-tuning does.

[4] You would be only graded by the correctness of your implementation, but if you want good performance, Try using a larger learning rate and longer training epochs.



Generated plot from the head-tuning run (`plots/3.2.2_roberta_head.png`).

3. **TODOs**: Write down the number of tuned parameters for our prefix-tuning on BoolQ that appends trainable prefix embeddings of length 128, given the model hidden size is d.

   Answer: $130d + 2$. The prefix is a $128 \times d$ matrix, which is $128d$ numbers, and prefix-tuning also trains the classifier head, which adds $2d + 2$ from the last part. The total is $128d + 2d + 2 = 130d + 2$. For RoBERTa-base that's $130(768) + 2 = 99{,}842$, and the script printed 99,842 for the prefix run.

   A note on the name. What this assignment builds (trainable vectors added only at the input embeddings) is closer to what Lester et al. (2021) call prompt tuning. Prefix-tuning as Li and Liang (2021) define it trains prefix vectors for the keys and values in every layer, which for RoBERTa-base with 12 layers and $\ell = 128$ would be $2 \times 12 \times 128 \times 768 = 2{,}359{,}296$ parameters.

- It's time to implement prefix tunning!
  TODOs: Read and complete the missing lines after `if self.type == 'prefix'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. You would also need to modify the optimizer by adding code after `mymodel.type == "prefix":` in the `train` function in `classification.py`. Report the final accuracy of prefix-tuning along with the hyper-parameters you used[5].

  Done in `classification.py`. The `prefix` branch of `forward` copies the 128 × 768 prefix once per example with `expand`. That shares memory, and the gradients from every copy

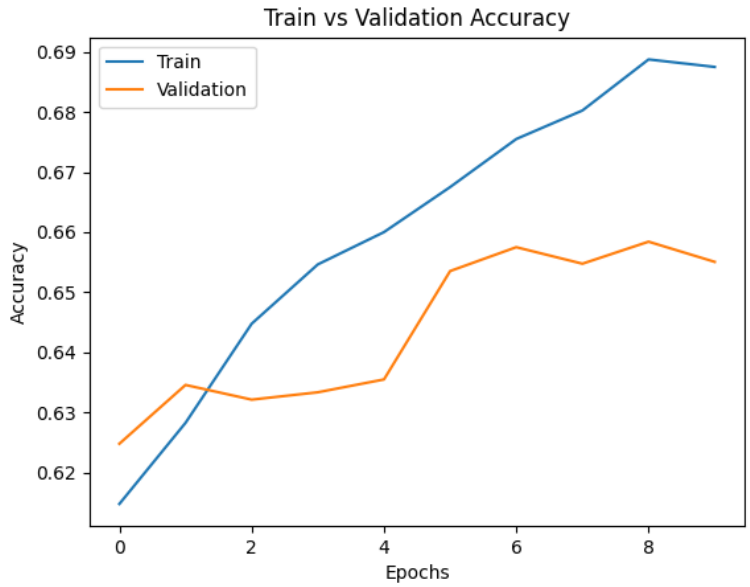
[5] Again, you would be only graded by the correctness of your implementation, but if you want good performance, Try using a larger learning rate and longer training epochs.

add up in the one `self.prefix`. It looks up the word embeddings of the 128 input tokens and puts the prefix in front, which gives (batch, 256, 768). The attention mask gets 128 ones in front, so every token can attend to the prefix. That goes into RoBERTa through `inputs_embeds`, and RoBERTa still adds its position embeddings and applies its embedding LayerNorm. The output is averaged over all 256 positions and goes through the classifier, as in the other modes. In `train()`, I freeze the encoder and build `custom_optimizer`, an AdamW over the prefix and the classifier's weight and bias. Gradients still flow back through the frozen encoder to reach the prefix, but none are stored for the encoder's weights.

One fix outside the TODOs, which affects 3.2.2 too. The skeleton attached the learning-rate scheduler to `optimizer` in every mode, but head-tuning and prefix-tuning step `custom_optimizer`. Their learning rate would have stayed fixed for the whole run, with no 50-step warm-up and no linear decay. I attached the scheduler to whichever optimizer actually steps.

Answer: dev accuracy 0.6550 and test accuracy 0.6580, with learning rate 5e-3, 10 epochs, and batch 16, in bfloat16 mixed precision (`--amp`). Following the footnote again, I used 5 times head-tuning's learning rate and the same 10 epochs. Batch 16 and `--amp` were for memory, which 3.2.4 explains. The script printed 99,842 tuned parameters. Dev accuracy went from 0.625 after epoch 1 to a high of 0.658 after epoch 9 and ended at 0.655. Training accuracy ended at 0.688. The run took 47.5 minutes on my GPU.

That's 1.3 points above head-tuning on dev (0.655 against 0.643) and 1.9 points on test (0.658 against 0.639). It's still 13 points below full fine-tuning on dev. Training accuracy tells the same story as in 3.2.2. It reached 0.688, against 0.658 for head-tuning and 0.964 for full fine-tuning. The prefix gives every layer 128 extra positions to attend to, so it can change what the frozen encoder computes, which a head on top can't do. But the frozen weights still do all the processing, and 99,842 numbers at the input only moved them a little. Dev was still edging up when the learning rate decayed to zero, so more epochs might add a point. I don't think they'd close the gap. Lester et al. (2021) found that prompt tuning only matches full fine-tuning once T5 passes billions of parameters (their largest has 11 billion), and RoBERTa-base has 125 million. They also found that starting the prompt from real vocabulary or class-label embeddings beats a random start at the smaller sizes. The skeleton starts the prefix as standard normal noise.

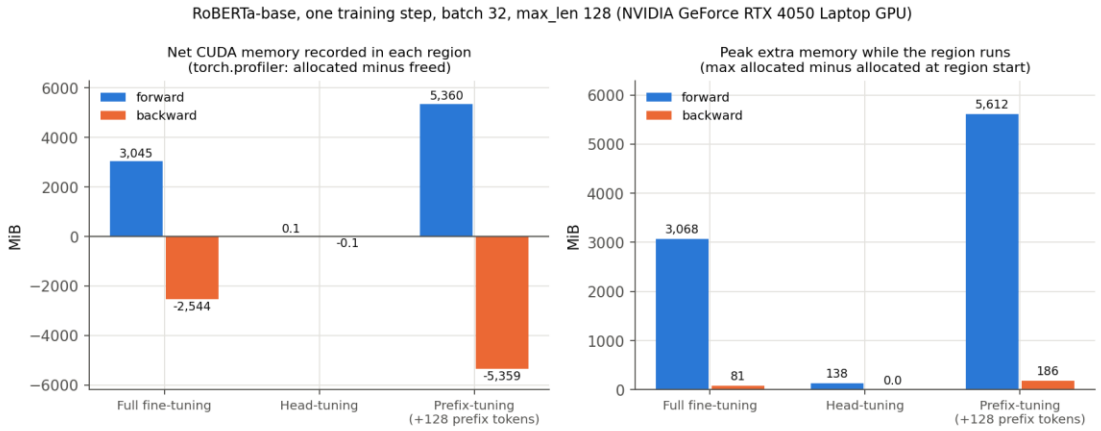


Generated plot from the prefix-tuning run (`plots/3.2.3_roberta_prefix.png`).

4. **Extra Credit:** Here is a simple tutorial on how to use torch.profile to record the memory usage of forward and backward operations.

- **TODOs** Generate a bar plot of the forward/backward memory usage of full-finetuning, head-tuning, and prefix-tuning. Note that you don't need to upload any code for this.

  I measured one training step of each method on RoBERTa-base with a real batch of 32 BoolQ training examples (max_len 128, prefix length 128), using `CustomModelforSequenceClassification` and the same freezing and optimizers as `train()`. One warm-up step runs first, so AdamW's state already exists. The measured step runs inside `torch.profiler.profile(profile_memory=True)`, with the model call and loss wrapped in `record_function("forward")` and `loss.backward()` wrapped in `record_function("backward")`, as in the PyTorch profiler recipe. The left panel is what the profiler records for each region, which is memory allocated minus memory freed inside it. The right panel is the peak extra memory during each region, from `torch.cuda.max_memory_allocated()`. The script is `experiments/profile_memory.py`, and the numbers are in `results/memory_profile.json`.



Forward and backward GPU memory for one training step (`plots/3.2.4_memory_profile.png`).

	Full fine-tuning	Head-tuning	Prefix-tuning
Forward, net (MiB)	3,045	0.1	5,360
Forward, peak extra (MiB)	3,068	138	5,612
Backward, net (MiB)	-2,544	-0.1	-5,359
Backward, peak extra (MiB)	81	0.0	186
Held between steps: weights and AdamW state (MiB)	1,484	494	495

Full fine-tuning saves 3,045 MiB of activations in the forward pass, because every layer's weights need a gradient, and each of those gradients needs that layer's inputs. The backward pass frees those activations and allocates gradients for the 124.06 million weights that train (about 473 MiB at 4 bytes each), so it nets out at 3,045 - 473 = 2,572 MiB freed, close to the 2,544 recorded. Its fixed cost is also the largest, 1,484 MiB, because AdamW keeps two extra numbers for every weight that trains. That's about 476 MiB of weights plus about 947 MiB of AdamW state, and I didn't track down the last 60 MiB or so. Gradients aren't part of it, because `zero_grad(set_to_none=True)` frees them at the end of each step.

Head-tuning needs almost nothing. Nothing before the classifier requires a gradient, so autograd keeps no activations inside RoBERTa. Each layer's output is freed as soon as the next layer has used it, which is the 138 MiB transient peak. The only thing saved is the averaged 32 × 768 representation the classifier needs, about 0.1 MiB, and the backward pass only touches the 1,538 head parameters.

Prefix-tuning was the surprise. It trains about 100,000 parameters, so its fixed cost matches head-tuning, but its forward pass saves 5,360 MiB, 76% more than full fine-tuning. The gradient has to flow back through all 12 frozen layers to reach the prefix, so most of each layer's activations still have to be kept. The attention scores, the softmax outputs, and the inputs to GELU and LayerNorm are all needed to pass the gradient back.

The inputs to the frozen linear layers aren't, because a frozen weight needs no gradient. On top of that, the prefix doubles the sequence from 128 to 256 positions, which doubles most activations and quadruples the attention score matrices. Its backward pass frees almost exactly what the forward saved, because the only gradients it stores are for the prefix and the head (about 0.4 MiB). Peak use for one step is roughly 1,484 + 3,068 = 4,552 MiB for full fine-tuning and 495 + 5,612 = 6,107 MiB for prefix-tuning. The card has 6,141 MiB in total, and part of that goes to the CUDA context, so prefix-tuning at batch 32 doesn't fit in float32. I trained it at batch 16 with `--amp`, which stores most activations in bfloat16 at about half the size. (Batch 32 in bfloat16 still spilled into shared system memory.) So prefix-tuning saves gradient and optimizer memory, but at this prefix length it costs more activation memory than it saves.

5. **LoRA** fine-tuning decomposes the parameter updates of a large dense matrix to two low-rank matrices. For a matrix M of size $\mathbb{R}^{d\times d}$, the full updates require storing the gradients and optimizer states of $d \times d$ parameters.
   TODOs: However, if we decompose the gradient update on the full matrix $\Delta M \approx AB$ where $A \in \mathbb{R}^{d\times n}$ and $B \in \mathbb{R}^{n\times d}$, write down the number of parameters whose gradients and optimizer states we need to store.

   Answer: $2dn$. A has $d \times n$ entries and B has $n \times d$, so the trainable count is $dn + nd = 2dn$, against d^2 for updating M directly. Each of those parameters needs a gradient and, with Adam, two moment estimates. The ratio is $2dn/d^2 = 2n/d$, so LoRA saves memory whenever $n < d/2$, and in practice n is far smaller. For one 768×768 attention matrix in RoBERTa-base with $n = 8$, that's $2(768)(8) = 12{,}288$ parameters instead of $589{,}824$, about 2.1%. M itself stays frozen, so it still takes memory for its weights, just not for gradients or optimizer state.

6. **Extra Credit:** Implement **LoRA** fine-tuning.
   TODOs: Create `classification_lora.py` and copy `classification_base.py` to it and implement the LoRA fine-tuning method using the peft library. This should be just changing a few lines of code. Report the trainable parameters and final accuracy of your LoRA fine-tuning, as well as your hyper-parameters.

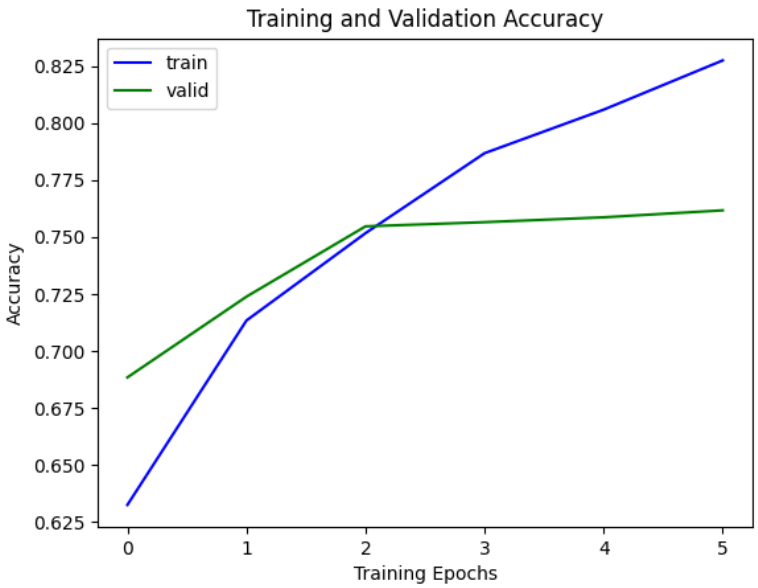
   Done in `hw6/classification_lora.py`. The assignment says to copy `classification_base.py`, and I took that to mean `base_classification.py`, since there's no file by the first name. The LoRA change is a few lines. After loading `AutoModelForSequenceClassification`, the script wraps the model with peft's `get_peft_model` and a `LoraConfig` with `task_type=SEQ_CLS`, rank $n = 8$, `lora_alpha` 16, and dropout 0.1. For RoBERTa, peft puts LoRA on the query and value projections of every attention layer by default, the same two matrices Hu et al. (2022) used. `SEQ_CLS` also keeps the classification head trainable, which it has to be, since the head starts from random weights. The optimizer gets only the parameters that require gradients. The rank, alpha, and dropout are command-line options (`--lora_r`, `--lora_alpha`, `--lora_dropout`).

   Answer: 887,042 trainable parameters, 0.71% of the 125,534,212 in the wrapped model. The script splits that count in two. 294,912 are the LoRA A and B matrices. That's $2dn =$

$2(768)(8) = 12{,}288$ from 3.2.5 for each of 24 matrices (query and value in 12 layers). The other 592,130 are RoBERTa's classification head, a 768 × 768 dense layer and a 768 × 2 output layer, with biases. So the head trains twice as many parameters as LoRA does.

Final accuracy: dev 0.7618 and test 0.7540, with learning rate 5e-4, 6 epochs, batch 32, and bfloat16 mixed precision (`--amp`). The run took 13.2 minutes on my GPU, with 2.33 GB peak reserved. Dev accuracy after each epoch was 0.688, 0.724, 0.755, 0.757, 0.759, and 0.762, and training accuracy ended at 0.828. I took the learning rate from Hu et al. (2022), who used 4e-4 to 5e-4 for RoBERTa-base with rank 8. That's 25 times the 2e-5 I used for full fine-tuning. LoRA needs the bigger steps because B starts at zero, so the update AB starts at zero and has to grow from there. They also trained for 25 to 80 epochs depending on the task, and I trained for 6.

LoRA lands much closer to full fine-tuning than the other two cheap methods. On dev it's 2.4 points below full fine-tuning from 3.2.1 (0.762 against 0.786), and 10.7 and 11.9 points above prefix-tuning and head-tuning. Its training accuracy (0.828) also shows it can fit the data, which head-tuning (0.658) and prefix-tuning (0.688) couldn't. The difference is where the trainable parameters sit. LoRA changes the query and value weights inside every layer, so it changes how each layer processes its input. The prefix can only add inputs for the frozen layers to attend to. The 3.2.1 comparison mixes in a different head, though. The 2e-5 RoBERTa-base run from 3.1.4 uses the same `AutoModelForSequenceClassification` head as the LoRA script, and LoRA matches it: 0.762 against 0.764 on dev, and 0.754 against 0.748 on test, while training 0.71% of the parameters. The settings still differ (6 epochs on my laptop for LoRA, 9 on the Colab L4 for full fine-tuning, both in bfloat16), so I'd call that a tie, not a win.



Generated plot from the LoRA run (`plots/3.2.6_roberta_lora.png`).

In-Context Learning

A note before 3.3. The OpenAI models this section names can't be called anymore. OpenAI shut down `davinci` on January 4, 2024, and `gpt-3.5-turbo-instruct`, `davinci-002`, and `babbage-002` on September 28, 2026, according to its API deprecations page, and the Playground's Completion mode went with them. So I ran every part of 3.3 on a local pair of open models that play the same two roles. Qwen2.5-1.5B is a base model with pretraining only, so it stands in for `davinci`, the "vanilla language model (no additional tuning with human feedback)." Qwen2.5-1.5B-Instruct is the same model after instruction tuning (Qwen Team, 2024), so it stands in for `gpt-3.5-turbo-instruct`. Both run on my GPU in bfloat16 as plain text completion, with no chat template, which is what the Completions API did. Decoding is greedy (temperature 0), so a rerun gives the same text. The scripts are `hw6/playground_prompts_local.py` and `hw6/openai_gpt-boolq.py`. The second still has the original OpenAI code path behind `--backend openai`. Keep in mind that these models have 1.5 billion parameters, roughly a hundred times fewer than GPT-3 davinci's 175 billion, so their numbers say something about the method at a small scale, not about GPT-3.5.

1. Here we will prompt language models.[6] You should sign up for the OpenAI API, which lets you use GPT-3 a large, neural language model like the ones that we learned about in lecture.

- The OpenAI API is a paid service. OpenAI will give you a few dollars in credit when you first create your account (See the usage page). For this assignment, the cost should be less than that. For the first part of the assignment, we'll get warmed up by playing with the OpenAI API via its interactive Playground website. Later we'll see how to integrate it directly into our code.

OpenAI

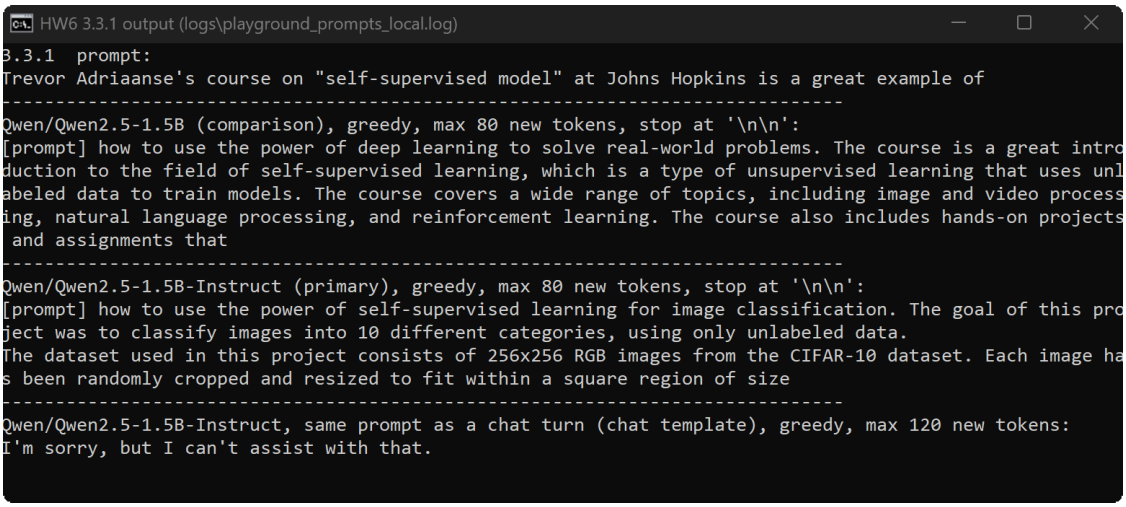
- OpenAI products offer a wide spectrum of capabilities and modes of interaction with their models. For this homework, **we use the `Completion` mode for both playground and API calls**. In the future, you can explore other modes that might be more up-to-date and powerful, e.g. `Chat`(Playground, APIs), `Assistants` (Playground, APIs), and many more!
  Note: we suggest you to prioritize your credit usage on finishing this homework question with `Completion` before exploring other modes, as querying powerful models might be more expensive.
  First, let's learn some basic terminology:
  - Prompt: the input to the model
  - Completion: what the model outputs

[6] Question credit: Chris Callison-Burch's AI course at UPenn.

Let's try it out.
TODOs: Paste this prompt into the playground, press the "Generate" button, and see what it says. Paste a screenshot of the completion below:

```
Trevor Adriaanse's course on "self-supervised model" at Johns Hopkins
is a great example of
```

The Playground no longer exists, so this is a screenshot of the console output from `hw6/playground_prompts_local.py` instead. The same text is in `logs/playground_prompts_local.log` and `hw6/playground_prompts_local.json`. The prompt is exactly as given, decoding is greedy (temperature 0), and the limit is 80 new tokens, with a stop at a blank line. The answer is the middle block, from Qwen2.5-1.5B-Instruct, standing in for `gpt-3.5-turbo-instruct`. In each block, "[prompt]" stands for the prompt, and the text after it is the completion. The first block is the base model and the last is the same prompt as a chat turn, both discussed below.



Console output for 3.3.1 (`plots/3.3.1_terminal.png`).

It reads like a course description, and almost none of it is true. The course is about language models, not image classification, and CIFAR-10 images are 32 × 32 pixels, not 256 × 256. The model knows nothing about this course, so it writes the kind of text that usually follows "a great example of," with full confidence. It also stopped mid-sentence at the 80-token limit. The base model, run on the same prompt for comparison, made up its content too, a course covering "image and video processing, natural language processing, and reinforcement learning."

I also sent the prompt to the Instruct model as a chat turn, through its own chat template, which is the format its instruction tuning used (the script saves the exact chat prompt). There it answered "I'm sorry, but I can't assist with that." I don't know what triggered the refusal. My guess is a real person's name in a sentence fragment that doesn't ask for anything.

- Now save its output for the end of the semester for your course reviews ... just kidding!!

2. There are several controls on the right hand side of the playground. These are:

   - **Model:** GPT models come in several different sizes and capabilities. You can read more documentation on these models here. For our homework, we use the `gpt-3.5-turbo-instruct` model.
   - **Temperature and Top P sampling:** control how the model samples tokens from its distribution. Setting Temperature to 0 will cause the model to produce the highest probability output. Setting it closer to 1 will increase its propensity to create more diverse output.
   - **Maximum length:** what's the maximum length (in tokens) that the model will output?
   - **Stop sequences:** you can specify what tokens should cause the model to stop generating. You can use the newline character or any special sequence that you designate.
   - **Frequency Penalty and Presence Penalty:** two parameters that help to limit how much repetition there is in the model's output.
   - **Best of:** Best of n indicates the model generate n completion candidates on the server side, and display the top-1 on the client (user) side.

- For the remaining parts of this section, set the engine to `davinci` so that it uses vanilla language model (no additional tuning with human feedback).
  In-context learning: In addition to writing awesome reviews of your professors, you can design prompts to get GPT-3 to do all sorts of surprising things. For instance, GPT-3 can perform few-shot learning. Given a few examples of a task, it can learn a pattern very quickly and then be used for classification tasks. It often helps to tell the model what you want it to do.

  Here's an example from the paper that introduced GPT-3. It shows a few-show learning example for correcting grammatically incorrect English sentences.

```
Poor English input:  I eated the purple berries.
Good English output:  I ate the purple berries.

Poor English input:  Thank you for picking me as your designer.  I'd
appreciate it.
Good English output:  Thank you for choosing me as your designer.  I
appreciate it.

Poor English input:  The mentioned changes have done.  or I did the
alteration that you requested.  or I changed things you wanted and did
the modifications.
Good English output:  The requested changes have been made.  or I made
the alteration that you requested.  or I changed things you wanted and
```

made the modifications.

Poor English input: I'd be more than happy to work with you in another project.

TODOs: Paste this prompt into the playground, press the “Generate” button, and see what it says. Paste the completion below:

The base model, Qwen2.5-1.5B, standing in for `davinci`, completed it like this (greedy, up to 40 new tokens, with a stop at a blank line):

```
Good English output:  I would be more than happy to work with you in another project.
```

It picked up the format from the three examples, including the label and the two spaces after the colon, and it stopped at the blank line where the next example would start. But the fix misses the actual error. It expanded “I’d” to “I would” and kept “in another project,” where the preposition should be “on.” The Instruct model, given the same raw prompt, made the real fix: “I would be more than happy to work with you on another project.” So did the Instruct model as a chat turn. Three examples were enough for the base model to learn the shape of the task. Getting the correction itself right is where the instruction tuning showed.

3. **Instruction following:** In addition to few shot learning, GPT and other large language models do a pretty remarkable job in “zero-shot” scenarios. You can give them instructions in natural language and they may produce remarkably good output.

- For example, let’s try the following prompt:

Correct this English text: Today I have went to the store to to buys some many bottle of water.

TODOs: Paste this prompt into the playground, press the “Generate” button, and see what it says. Paste the completion below:

The base model, Qwen2.5-1.5B, completed it like this (greedy, up to 60 new tokens, the prompt and then the completion):

```
Correct this English text:  Today I have went to the store to to buys some many bottle of water. I have bought 10 bottles of water and 5 bottles of juice. I have paid 10 dollars for the water and 5 dollars for the juice. I have got 15 dollars back from the store. I have went to the store to buy some many bottle of water.
```

It didn’t correct anything. It treated the instruction as the first line of a story and kept writing about bottles of water, and its last sentence repeats the original errors. A base model is only trained to predict the next token, and nothing in that training marks “Correct this English text:” as a command.

The Instruct model did no better as raw text. It continued the story too: “The price was very high, so I decided not to buy any.” Its instruction tuning only kicked in when the

prompt came as a chat turn, through its chat template. Then it answered “Today, I went to the store to buy several bottles of water.” That fixes every error and keeps the meaning, though “several” is its guess at what “some many” meant. So the model can follow the instruction, but only in the format it was tuned on.

4. **Programmatic access:** In addition to prompting GPT models in the playground, you can write Python code to query models with provided OpenAI APIs. Now let’s use few-shot prompting to solve a few instance of BoolQ.
   TODOs: Write a Python script for few-shot prompting OpenAI GPT models on BoolQ. Specifically, form a prompt by randomly selecting 8 demonstrations from BoolQ. Make sure to have a balanced prompt (50% ’yes’s and the rest ’no’s) and interleave them to prevent the recency and label-imbalance biases. Using this in-context demonstration, evaluate `gpt-3.5-turbo-instruct` on a small subset of BoolQ instances (say, 30 examples). Hand in your code used to perform this evaluation, it should be named as `openai_gpt-boolq.py`. Write down the evaluation accuracy below.
   Hints:

   – We use the `gpt-3.5-turbo-instruct` model (`model="gpt-3.5-turbo-instruct"`)

   – You can use the playground to create a skeleton code to start with based on a prompt that you can then use in your Python projects. Click on the “View Code” button on the top-right, and you’ll get some code that you can convert into a script.

   – You can also refer to the full documentation for creating completion and processing the response.

   – Using OpenAI API requests an API key. You can generate a key associated with your account here. To input the key, you can either do `export OPENAI_API_KEY=<your key>` on your compute environment before running the script. Or refer to the documentation.

   Done in `hw6/openai_gpt-boolq.py`, which started from the skeleton’s `gpt3_boolq_evaluation.py`. The prompt opens with one instruction line (“Read each passage and answer the question about it with yes or no.”), then 8 demonstrations from the BoolQ training split, each as `Passage:`, `Question:`, and `Answer: yes` or `Answer: no`. The script draws 4 yes and 4 no examples at random (seed 0) and interleaves them yes, no, yes, no, so neither label dominates the prompt or sits in a run at the end. It only draws from training passages of 120 words or fewer, to keep the prompt well inside `gpt-3.5-turbo-instruct`’s 4,096-token window. The 30 prompts ran 997 to 1,219 tokens with Qwen’s tokenizer. The test passage and question come last, ending in `Answer:`. Decoding is greedy, with at most 2 new tokens and a newline stop. The answer counts as right if it starts with the gold label after stripping and lowercasing. The skeleton compared the raw completion to “True” or “False” with ==, so a leading space would have counted as wrong. The 30 questions are a random sample of the validation split, drawn with the

same seed. The local backend also records the model's probability of " yes" against " no" as the next token after `Answer:`, which gives a forced-choice accuracy that can't have an unparseable answer. The OpenAI code path is still there behind `--backend openai`.

Answer: 21 of 30 correct, an accuracy of 0.700, with Qwen2.5-1.5B-Instruct standing in for `gpt-3.5-turbo-instruct`. Always answering yes would get 19 of 30 (0.633) on the same questions. With 30 questions, each one is worth 3.3 points, and a 95% interval around 0.700 runs from about 0.52 to 0.83 (Wilson). So I also ran the same prompt on all 3,270 validation questions. The Instruct model got 2,545 right, 0.778 (95% interval 0.764 to 0.792), against 0.622 for always-yes. It answered yes 2,067 times and no 1,201 times, close to the true split of 2,033 and 1,237. Two completions didn't start with yes or no ("Atlantic" and "north america"). My first parser read "north america" as a no because it starts with the letters n and o. That one was wrong either way, so the accuracy doesn't change, and `experiments/rescore_boolq.py` corrected the counts in the saved results. The run took 15.5 minutes on my GPU. Results and every completion are in `results/boolq_fewshot_*.json`, and the logs are in `logs/`.

For comparison, I ran the same prompt on the base model, Qwen2.5-1.5B, the stand-in for `davinci`. It got 19 of 30 (0.633), no better than always-yes on those questions. On all 3,270 it got 2,439 right, 0.746. It answered yes 1,710 times and no 1,557 times, against 2,033 and 1,237 true, so it leans toward no. One possible reason, which I didn't test: the demonstrations alternate yes, no, yes, no, so every prompt ends on a no example, and a model that copies the most recent label would drift toward no. The Instruct model doesn't lean that way. Three base-model completions didn't start with yes or no ("zone", "Caribbean is", and "Atlantic Ocean"). So instruction tuning added 3.2 points on the full set, 0.778 against 0.746. Most of the gain over always-yes (0.622) is already there in the base model, which says the eight demonstrations carry most of the task. The forced-choice accuracies tell the same story: 0.753 for the base model and 0.779 for Instruct.

The 0.778 is the number that surprised me. With no training at all, eight examples in the prompt get a 1.5-billion-parameter model to within a point of full fine-tuning RoBERTa-base in 3.2.1 (0.786 on the same 3,270 questions) and ahead of LoRA (0.762). It isn't a fair fight in either direction. The Qwen model is twelve times larger than RoBERTa-base and was pretrained on far more text, and its instruction tuning may have included question answering much like BoolQ. I can't rule out that BoolQ itself was in its training data. On the other side, the fine-tuned models saw 8,000 labeled examples, and the prompt has 8.